MANUAL TÉCNICO

TRAVELERS

Cómo está hecho el código y cómo funciona

Backend FastAPI · Frontend React · MySQL · AWS

Explicado paso a paso y con ejemplos del código real

Versión 1.1 · Noviembre de 2026

```python
# backend/app/routes/reservation_routes.py
@router.post("/", status_code=201)
def create_reservation(reservation_data: ReservationCreate,
                       db: Session = Depends(get_db),
                       current_user = Depends(get_current_user)):
    return ReservationService(db).create_reservation(
        reservation_data, current_user)
```

```javascript
// frontend/travelers/src/api/axiosConfig.js
const api = axios.create({ baseURL: API_URL });
api.interceptors.request.use((config) => {
  config.headers.Authorization = `Bearer ${token}`;
});
```

Proyecto Técnico en Programación de Software · SENA 2026

CONTENIDO

ÍNDICE

1. Antes de empezar 5

La idea en una frase 5

Una comparación: el restaurante 5

Palabras que vas a encontrar 6

2. Panorama del proyecto 7

Tecnologías y para qué sirve cada una 8

Cómo están organizadas las carpetas 9

Cómo se pone a funcionar en un computador 9

3. El backend por dentro 10

Las capas 11

Cómo arranca: main.py 11

La conexión a la base de datos 12

Los modelos: una clase por tabla 12

Los esquemas: el filtro de entrada 12

Los repositorios: ir a la despensa 13

4. Ejemplo completo: crear una reserva 14

Paso 1. La ruta recibe la petición 15

Paso 2. El servicio revisa las reglas 15

Paso 3. El cálculo del precio 17

Paso 4. Guardar y avisar 17

Paso 5. La respuesta 17

El truco de los errores 17

5. Seguridad: sesiones, roles y contraseñas 18

Las contraseñas nunca se guardan tal cual 18

El token JWT: la pulsera de acceso 18

get_current_user: ¿quién está pidiendo? 18

Los roles 19

Límite de intentos de inicio de sesión 19

La bitácora de administradores . . . 19

6. Otras piezas del backend . . . 21

Pagos con Stripe . . . 21

Cancelaciones y reembolsos . . . 22

Notificaciones: correo y campana . . . 22

Recordatorios automáticos . . . 22

Fotos: Amazon S3 y CloudFront . . . 23

Coordenadas automáticas de las ciudades . . . 23

Voucher con QR, reportes y configuración . . . 23

7. La base de datos . . . 24

Las 21 tablas agrupadas por tema . . . 25

Cómo se relacionan las principales . . . 25

Las migraciones . . . 25

8. El frontend por dentro . . . 26

Tres ideas de React para entender todo lo demás . . . 27

El punto de arranque: main.jsx . . . 27

Las pantallas: App.jsx . . . 27

ProtectedRoute: el portero . . . 29

Hablar con el backend: axiosConfig.js . . . 29

La sesión: AuthContext . . . 29

Una pantalla por dentro: el formulario de reserva . . . 30

Favoritos: respuesta inmediata . . . 30

Ayudantes reutilizables . . . 31

La consola de administración . . . 31

El buscador del catálogo . . . 31

Paquetes relacionados y compartir . . . 32

Carga más rápida . . . 32

Los estilos . . . 32

9. Recorrido completo: de un clic a la base de datos . . . 33

10. Receta: agregar una funcionalidad nueva . . . 34

11. Pruebas automáticas 35

Cómo funcionan 35

12. Cómo se publicó en internet 36

Cómo se actualiza 36

CAPÍTULO 1

ANTES DE EMPEZAR

Este manual explica cómo está construido Travelers por dentro. Está pensado para que cualquier persona, aunque esté empezando a programar, entienda qué hace cada parte del código y cómo se conectan entre sí.

La idea en una frase

Travelers tiene **dos programas** que conversan entre sí: el **frontend**, que es lo que el usuario ve en el navegador, y el **backend**, que guarda los datos y aplica las reglas del negocio. El backend guarda todo en una **base de datos MySQL**.



Una comparación: el restaurante

Pensemos en un restaurante. El **cliente** mira la carta y pide (frontend). El **mesero** lleva el pedido a la cocina y trae el plato (la API). El **chef** sabe las recetas y decide si el pedido se puede preparar (los servicios del backend). La **despensa** guarda los ingredientes (la base de datos).

En el restaurante	En Travelers	Dónde está en el código
El cliente y la carta	Las pantallas que ve el usuario	frontend/travelers/src/pages
El mesero	Las rutas del API (reciben y responden)	backend/app/routes
El chef y sus recetas	Los servicios (reglas del negocio)	backend/app/services
El ayudante que va a la despensa	Los repositorios (consultas a la base)	backend/app/repositories
La despensa	La base de datos MySQL	database/script_BD_Travelers.sql

Palabras que vas a encontrar

Palabra	Qué significa
API	La “puerta” del backend: una lista de direcciones (rutas) a las que el frontend puede pedir o enviar datos. Travelers tiene 122.
Petición HTTP	Un mensaje del navegador al servidor. Tiene un método (qué quiero hacer) y una ruta (sobre qué).
GET / POST / PATCH / DELETE	Los métodos: consultar, crear, modificar y borrar.
JSON	El formato de texto en que viajan los datos, por ejemplo {"cantidad_personas": 2}.
Código de estado	El número con que el servidor responde: 200 bien, 201 creado, 400 dato inválido, 401 sin sesión, 403 sin permiso, 404 no existe.
Token (JWT)	Una “pulsera de acceso” que el backend le da al usuario al iniciar sesión; se envía en cada petición para demostrar quién es.
ORM	Una herramienta (SQLAlchemy) que permite trabajar con tablas como si fueran clases de Python, sin escribir SQL a mano.
Componente	Una pieza de pantalla en React (un botón, una tarjeta, una página) escrita como una función.
Estado	Datos que una pantalla recuerda mientras está abierta (lo que el usuario escribió, lo que llegó del servidor).

CAPÍTULO 2

PANORAMA DEL PROYECTO

Qué tecnologías se usaron, cómo están organizadas las carpetas y cómo se pone a funcionar el proyecto en un computador.

Tecnologías y para qué sirve cada una

Tecnología	Parte	Para qué se usa
Python + FastAPI	Backend	Crear el API: recibir peticiones, validarlas y responder en JSON.
SQLAlchemy	Backend	Hablar con MySQL usando clases de Python (modelos).
Pydantic	Backend	Validar automáticamente los datos que llegan (tipos, mínimos, obligatorios).
passlib (bcrypt) y python-jose	Backend	Cifrar contraseñas y crear los tokens JWT.
reportlab y openpyxl	Backend	Generar vouchers y reportes en PDF y Excel.
boto3	Backend	Subir fotos a Amazon S3.
Stripe (API REST con requests)	Backend	Cobrar con tarjeta en modo de prueba y hacer reembolsos.
pytest	Backend	Pruebas automáticas (238 pruebas).
React 19	Frontend	Construir la interfaz con componentes.
React Router 7	Frontend	Cambiar de pantalla según la dirección (/packages, /admin...).
Axios	Frontend	Hacer las peticiones HTTP al backend.
Leaflet	Frontend	Mostrar el mapa de destinos.
Vite	Frontend	Servidor de desarrollo y compilación para producción.
MySQL 8	Datos	Guardar toda la información.
AWS (EC2, S3, CloudFront), Nginx	Despliegue	Publicar la aplicación en internet con HTTPS.

Cómo están organizadas las carpetas

Estructura principal

```
ProyectoTravelers/
├── backend/
│   ├── app/
│   │   ├── main.py          ← punto de arranque del API
│   │   ├── core/            ← seguridad, constantes, archivos, bitácora
│   │   ├── database/        ← conexión a MySQL
│   │   ├── models/          ← una clase por tabla (21)
│   │   ├── schemas/         ← cómo deben llegar y salir los datos
│   │   ├── repositories/    ← consultas a la base de datos
│   │   ├── services/        ← reglas del negocio (29 servicios)
│   │   └── routes/          ← las rutas del API (15 archivos)
│   ├── test/                ← pruebas automáticas
│   └── requirements.txt     ← librerías de Python
├── frontend/travelers/
│   └── src/
│       ├── main.jsx         ← punto de arranque de React
│       ├── App.jsx          ← lista de pantallas (rutas)
│       ├── api/             ← configuración de Axios
│       ├── context/         ← sesión y favoritos compartidos
│       ├── pages/           ← una carpeta por pantalla (14)
│       ├── components/      ← piezas reutilizables (27)
│       └── utils/           ← funciones de ayuda
└── database/                ← script SQL y 6 migraciones
```

Cómo se pone a funcionar en un computador

1. Crear la base de datos ejecutando **database/script_BD_Travelers.sql** y luego las migraciones en orden (001 a 006).
2. En **backend/**, crear el entorno virtual e instalar las librerías: `pip install -r requirements.txt`.
3. Crear el archivo **backend/.env** con los datos de conexión (DB_HOST, DB_USER, DB_NAME...) y la clave secreta de los tokens.
4. Arrancar el backend: `uvicorn app.main:app --reload`. Queda en http://127.0.0.1:8000 y la documentación automática en /docs.
5. En **frontend/travelers/**: `npm install` y luego `npm run dev`. La página abre en http://localhost:5173.

> **Consejo.** FastAPI genera sola una página de documentación en **/docs** donde se pueden ver y probar las 122 rutas del API sin escribir código.

CAPÍTULO 3

EL BACKEND POR DENTRO

El backend está organizado en capas. Cada capa tiene un solo trabajo y solo habla con la capa de al lado. Así el código es más fácil de leer, probar y cambiar.

Las capas

Ruta recibe y responde › **Esquema** valida los datos › **Servicio** aplica las reglas › **Repositorio** consulta la base › **Modelo** representa la tabla

Capa	Su único trabajo	Ejemplo
Ruta (routes)	Define la dirección y el método, pide lo que necesita y convierte los errores en códigos HTTP.	POST /reservations/
Esquema (schemas)	Describe qué campos deben llegar y salir y de qué tipo son.	ReservationCreate
Servicio (services)	Contiene las reglas: ¿hay cupo?, ¿la fecha es válida?, ¿cuánto cuesta?	ReservationService
Repositorio (repositories)	Guarda, busca y actualiza registros. No decide nada.	ReservationRepository
Modelo (models)	Una clase de Python que corresponde a una tabla de MySQL.	ReservationModel → tabla reservas

Cómo arranca: main.py

Cuando se ejecuta `uvicorn app.main:app`, Python lee **main.py**. Ese archivo crea la aplicación, le agrega los "filtros" que revisan cada petición (middlewares) y conecta los 15 archivos de rutas:

backend/app/main.py (resumido)

```
app = FastAPI(title="Travelers API", version="1.0.0", lifespan=lifespan)

# Bitácora de acciones de administradores
app.add_middleware(AuditMiddleware)

# Qué páginas web pueden llamar al API
app.add_middleware(CORSMiddleware, allow_origins=origins, ...)

app.include_router(user_router)
app.include_router(reservation_router)
app.include_router(payment_router)
# ... 14 routers en total
```

- **lifespan**: lo que se hace al encender y apagar el servidor. Aquí se inicia una tarea que cada hora revisa si hay recordatorios de viaje por enviar.
- **CORS**: el navegador bloquea por seguridad las peticiones a otro dominio. Con esta configuración el backend le dice al navegador qué páginas sí pueden llamarlo.
- **include_router**: cada archivo de rutas agrupa las direcciones de un tema (usuarios, reservas, pagos...) y aquí se suman a la aplicación.

La conexión a la base de datos

El archivo **database/connection.py** lee los datos de conexión del archivo .env y crea el "motor" que habla con MySQL. La función `get_db` abre una sesión para cada petición y la cierra al terminar, aunque haya un error:

backend/app/database/connection.py

```
engine = create_engine(DATABASE_URL, pool_pre_ping=True)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

def get_db():
    db = SessionLocal()      # abre la sesión
    try:
        yield db             # la presta a la ruta
    finally:
        db.close()           # y siempre la cierra
```

> **Nota. pool_pre_ping=True** hace que, antes de usar una conexión guardada, se compruebe que sigue viva. Así se evitan errores cuando MySQL cierra conexiones que llevan mucho tiempo sin usarse.

Los modelos: una clase por tabla

Cada modelo hereda de `Base` y declara sus columnas. SQLAlchemy usa esta información para leer y escribir en la tabla sin que haya que escribir SQL:

backend/app/models/reservation_model.py (resumido)

```
class ReservationModel(Base):
    __tablename__ = "reservas"

    id_reserva = Column(Integer, primary_key=True, autoincrement=True)
    fecha_reserva = Column(Date, nullable=False)
    cantidad_personas = Column(Integer, nullable=False, default=1)
    cliente_id = Column(Integer, ForeignKey("clientes.id_cliente"))
    id_paquete_turistico = Column(Integer,
        ForeignKey("paquetes_turisticos.id_paquete_turistico"))
    total_reserva = Column(DECIMAL(10, 2), nullable=False)
    estado = Column(String(45), default="pendiente")
    codigo_reserva = Column(String(45), unique=True)
```

`ForeignKey` indica la relación con otra tabla: cada reserva pertenece a un cliente y a un paquete. `DECIMAL(10, 2)` guarda el dinero con dos decimales exactos (un número de tipo float podría redondear mal).

Los esquemas: el filtro de entrada

Antes de que el código del servicio vea los datos, Pydantic los revisa con el esquema. Si falta un campo o tiene un tipo equivocado, FastAPI responde solo con un error **422** y la petición no avanza:

backend/app/schemas/reservation_schema.py

```
class ReservationCreate(BaseModel):
    id_paquete_turistico: int = Field(..., gt=0)    # obligatorio y > 0
    fecha_reserva: date                             # debe ser una fecha
    cantidad_personas: int = Field(..., gt=0)
    acompanantes: List[CompanionCreate] = []        # opcional
    codigo_cupon: Optional[str] = Field(None, max_length=30)
```

También hay esquemas de salida (`ReservationResponse`) que deciden qué campos se devuelven. Así nunca se envía por error un dato privado, como la contraseña cifrada de un usuario.

Los repositorios: ir a la despensa

Un repositorio solo sabe guardar y buscar. Si algo falla al guardar, deshace los cambios con `rollback` para no dejar la base a medias:

backend/app/repositories/reservation_repository.py (resumido)

```
class ReservationRepository:
    def __init__(self, db: Session):
        self.db = db

    def create_reservation(self, reservation):
        try:
            self.db.add(reservation)
            self.db.commit()           # guarda en MySQL
            self.db.refresh(reservation)  # trae el id generado
            return reservation
        except Exception as error:
            self.db.rollback()         # deshace si algo falló
            raise error

    def find_by_code(self, codigo_reserva):
        return (self.db.query(ReservationModel)
                .filter(ReservationModel.codigo_reserva == codigo_reserva)
                .first())
```

CAPÍTULO 4

EJEMPLO COMPLETO: CREAR UNA RESERVA

La mejor forma de entender el backend es seguir una petición de principio a fin. Este es el camino que recorre una reserva desde que llega hasta que se guarda.

Paso 1. La ruta recibe la petición

backend/app/routes/reservation_routes.py

```
@router.post("/", response_model=ReservationResponse, status_code=201)
def create_reservation(
    reservation_data: ReservationCreate,                  # el JSON validado
    db: Session = Depends(get_db),                        # la sesión de MySQL
    current_user: UserModel = Depends(get_current_user)   # quién está pidiendo
):
    try:
        return ReservationService(db).create_reservation(
            reservation_data, current_user)
    except PermissionError as error:
        raise HTTPException(status_code=403, detail=str(error))
    except LookupError as error:
        raise HTTPException(status_code=404, detail=str(error))
    except ValueError as error:
        raise HTTPException(status_code=400, detail=str(error))
```

Depends es la "inyección de dependencias" de FastAPI: antes de ejecutar la función, FastAPI llama a `get_db` y a `get_current_user` y le entrega los resultados. Si el token no es válido, `get_current_user` corta la petición con un 401 y la función ni siquiera se ejecuta.

Paso 2. El servicio revisa las reglas

El servicio es una lista de comprobaciones. Cada una tiene un nombre que dice qué revisa y lanza un error con un mensaje claro si no se cumple:

backend/app/services/reservation_service.py (resumido)

```
def create_reservation(self, reservation_data, current_user):
    client = self._get_client_profile_or_raise(current_user)
    tourist_package = self._get_package_or_raise(reservation_data.id_paquete_turistico)

    self._ensure_package_is_active(tourist_package)
    self._ensure_reservation_date_is_valid(reservation_data, tourist_package)
    self._ensure_valid_companion_count(reservation_data)
    self._ensure_package_has_capacity(tourist_package,
                                      reservation_data.cantidad_personas)

    total_reserva = self._calculate_total(tourist_package,
                                          reservation_data.cantidad_personas)
    # ... aplica el cupón si viene uno ...

    reservation = ReservationModel(..., estado="pendiente",
                                   codigo_reserva=self._generate_reservation_code())
    created = self.reservation_repository.create_reservation(reservation)

    self.companion_service.create_many_companions(...)       # acompañantes
    self.notification_email_service.create_reservation_confirmation_email(...)
    self.app_notifications.notify(...)                       # campana
    return created
```

Comprobación	Qué revisa	Si falla
_get_client_profile_or_raise	Que el usuario tenga perfil de cliente.	403
_get_package_or_raise	Que el paquete exista.	404
_ensure_package_is_active	Que el paquete esté activo.	403
_ensure_reservation_date_is_valid	Que la fecha sea desde mañana y esté dentro de las fechas del paquete.	400
_ensure_valid_companion_count	Que no haya más acompañantes que personas menos uno (el titular).	400
_ensure_package_has_capacity	Que queden cupos: cupo máximo menos las personas ya reservadas (sin contar canceladas).	400

Importante. Todas las comprobaciones se hacen **antes** de guardar. Si se guardara primero y luego fallara la validación de acompañantes, quedaría una reserva “huérfana” en la base. Hay una prueba automática que verifica justamente esto.

Paso 3. El cálculo del precio

backend/app/services/reservation_service.py

```
def _calculate_total(self, tourist_package, cantidad_personas):
    precio = Decimal(tourist_package.precio)
    descuento = Decimal(tourist_package.descuento or 0)
    subtotal = precio * cantidad_personas
    if descuento > 0:
        return subtotal - subtotal * (descuento / Decimal(100))
    return subtotal
```

Ejemplo: un paquete de $1.200.000 con 10 % de descuento para 2 personas: 1.200.000 × 2 = 2.400.000; menos el 10 % (240.000) = **$2.160.000**. Si además hay un cupón, su valor se resta de ese total.

Paso 4. Guardar y avisar

Con todo validado, se crea el objeto `ReservationModel` con estado **pendiente** y un código único como `RES-4F9A1C2B7D` (generado con uuid). El repositorio lo guarda, luego se guardan los acompañantes, se registra el correo de confirmación y se crea la notificación de la campana.

Paso 5. La respuesta

La ruta devuelve el objeto creado. FastAPI lo convierte en JSON usando `ReservationResponse` y responde con el código **201 Created**. El frontend recibe ese JSON y lleva al usuario a “Mis reservas”.

El truco de los errores

Los servicios no saben nada de HTTP: solo lanzan errores normales de Python. La ruta los traduce. Así el mismo servicio se podría usar desde otro lugar (por ejemplo, una prueba) sin cambiarlo:

Error de Python en el servicio	Código HTTP	Significado para el usuario
ValueError	400	El dato no cumple una regla (fecha, cupo, valor).
PermissionError	403	No tienes permiso para hacer esto.
LookupError	404	Lo que buscas no existe.
Error de validación de Pydantic	422	Falta un campo o tiene un tipo equivocado.
Token ausente o vencido	401	Debes iniciar sesión otra vez.

CAPÍTULO 5

SEGURIDAD: SESIONES, ROLES Y CONTRASEÑAS

Cómo sabe el sistema quién es cada usuario, qué puede hacer y cómo se protegen sus datos.

Las contraseñas nunca se guardan tal cual

Al registrarse, la contraseña pasa por **bcrypt**, que la convierte en un texto ilegible (hash). Para iniciar sesión no se "descifra": se vuelve a calcular y se compara.

backend/app/core/security.py

```
password_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

def hash_password(password):
    return password_context.hash(password)

def verify_password(password, hash_password):
    return password_context.verify(password, hash_password)
```

El token JWT: la pulsera de acceso

Cuando el usuario inicia sesión correctamente, el backend crea un **token JWT**: un texto firmado con una clave secreta que solo conoce el servidor. Dentro lleva el id del usuario (`sub`), la fecha de vencimiento (`exp`, 60 minutos) y para qué sirve (`purpose`).

backend/app/core/security.py

```
def create_access_token(data):
    payload = data.copy()
    payload["exp"] = datetime.now(timezone.utc) + timedelta(minutes=60)
    payload["purpose"] = "access"
    return jwt.encode(payload, SECRET_KEY, algorithm=ALGORITHM)
```

Se usan tres tipos de token con el mismo mecanismo, cada uno con su "purpose": **access** (sesión), **email_verification** (enlace para verificar el correo, 120 minutos) y **password_reset** (enlace para cambiar la contraseña, 30 minutos). Así un enlace de recuperación no sirve para iniciar sesión.

get_current_user: ¿quién está pidiendo?

Las rutas protegidas piden `Depends(get_current_user)`. Esta función lee el token del encabezado **Authorization: Bearer ...** y hace cuatro comprobaciones:

1. Que la firma sea válida y el token no esté vencido. Si no: **401**.
2. Que sea un token de tipo **access**. Si no: **401**.

3. Que el usuario exista en la base de datos. Si no: **401**.
4. Que la cuenta esté **activa**. Si el administrador la desactivó: **403**.

Los roles

Cada usuario tiene un `rol_id`. Las constantes están en **core/constants.py** para no repetir números sueltos por el código:

rol_id	Rol	Qué puede hacer
1	Administrador	Todo: la consola de administración, reportes, usuarios, pagos a proveedores.
2	Cliente	Reservar, pagar, cancelar, favoritos, reseñas.
3	Proveedor	Su panel: productos, órdenes de pago, datos de la empresa.

La revisión del rol se hace en el **backend** (por ejemplo `_ensure_admin` en cada servicio). El frontend también oculta pantallas según el rol, pero eso es solo comodidad: la seguridad real está en el servidor, porque cualquiera podría enviar peticiones sin usar la página.

Límite de intentos de inicio de sesión

Para frenar a quien intenta adivinar contraseñas (fuerza bruta), `core/login_limiter.py` cuenta los intentos fallidos:

- **Por correo**: después de 5 fallos en 15 minutos, ese correo queda bloqueado 15 minutos, aunque luego escriba la contraseña correcta.
- **Por dirección IP**: un límite de 20 fallos, para frenar a quien prueba muchos correos distintos.
- Al bloquear, el API responde **429 Too Many Requests** con el mensaje “Demasiados intentos fallidos. Intenta de nuevo en 15 minutos”.
- Un inicio de sesión correcto borra los fallos de ese correo.

> **Nota.** Los contadores viven en memoria porque el servidor corre con un solo proceso (`uvicorn --workers 1`). Con varios procesos habría que guardarlos en un lugar compartido, como Redis.

La bitácora de administradores

AuditMiddleware es un filtro que ve pasar todas las peticiones. Cuando un administrador hace un cambio (POST, PATCH, PUT o DELETE) y sale bien, busca la ruta en una lista de reglas y guarda una frase legible en la tabla **bitacora_admin**:

backend/app/core/audit_middleware.py (resumido)

```
RULES = [
  ("PATCH", r"^/packages/(\d+)/status$", "cambiar_estado", "paquete",
   lambda m, b: f"Cambió el estado del paquete #{m[1]} a {b['estado']}"),
  ("PATCH", r"^/users/(\d+)/role$", "cambiar_rol", "usuario",
   lambda m, b: f"Cambió el rol del usuario #{m[1]} a Proveedor"),
  # ...
]
```

Cada regla tiene una **expresión regular** que reconoce la ruta y una función que arma la descripción con los números de la ruta y los datos enviados.

CAPÍTULO 6

OTRAS PIEZAS DEL BACKEND

Además de las reservas, el backend tiene servicios para pagos, cancelaciones, avisos, archivos, mapas y documentos. Esta es la idea de cada uno.

Pagos con Stripe

Los pagos se hacen con **Stripe Checkout**, la página de pago de Stripe, en **modo de prueba**: el flujo es igual al real pero no se mueve dinero y solo se aceptan tarjetas de prueba (4242 4242 4242 4242). El código está en `stripe_service.py` y `stripe_routes.py`.

1. Checkout el backend crea la sesión › **2. Stripe** el viajero paga allá › **3. Vuelve** ?stripe_session=... › **4. Confirmar** el backend consulta a Stripe

1. **Crear la sesión** (`POST /payments/stripe/checkout`): el backend revisa que la reserva sea del viajero, esté pendiente y no tenga pago, y le pide a Stripe un cobro por el valor exacto en pesos. Stripe devuelve la dirección de su página de pago.
2. **Pagar**: el viajero escribe la tarjeta en la página de Stripe. Travelers nunca ve ni guarda los datos de la tarjeta.
3. **Volver**: Stripe devuelve al viajero a `/reservations?stripe_session=`.
4. **Confirmar** (`POST /payments/stripe/confirm`): el backend consulta esa sesión en Stripe y solo si está **pagada**, en pesos y por el valor exacto, confirma la reserva.
5. **Webhook** (`POST /payments/stripe/webhook`): Stripe también avisa al backend con un mensaje firmado (HMAC-SHA256 con el secreto whsec_...). Así la reserva se confirma aunque el viajero cierre la ventana antes de volver.

Las dos vías (volver y webhook) terminan en `process_session`, que no registra un pago dos veces. El pago aprobado pasa por `PaymentService.register_approved_payment`, la misma función del pago simulado:

Pago aprobado tabla pagos › **Reserva confirmada** estado = confirmada › **Órdenes de pago** una por proveedor › **Avisos** correo + campana

> **Nota.** Si en el .env no está `STRIPE_SECRET_KEY`, el sitio usa un **pago simulado** (se elige el método y se aprueba al instante). Así el proyecto funciona aunque no haya llaves de Stripe.

Las **órdenes de pago a proveedores** registran lo que la agencia le debe a cada proveedor cuyos productos (hotel, transporte, tour...) forman parte del paquete. El administrador las marca como pagadas desde la consola.

Cancelaciones y reembolsos

La regla vive en `_cancellation_quote`. Por defecto se puede cancelar sin costo hasta **7 días** antes del viaje; después se cobra una penalidad del **30 %** sobre lo pagado. Ambos valores se cambian desde la consola (tabla configuraciones).

Situación (pagó $1.000.000, viaje el 20 de dic.)	Penalidad	Reembolso
Cancela el 10 de dic. (antes del límite, 13 de dic.)	$0	$1.000.000
Cancela el 16 de dic. (después del límite)	$300.000	$700.000
Reserva pendiente (no pagó nada)	$0	$0
Cancela el 20 de dic. (el día del viaje)	No se puede cancelar	—

Si la reserva se pagó con Stripe, `_refund_in_stripe` devuelve el dinero **en Stripe** por el valor del reembolso (total o descontando la penalidad) **antes** de guardar la cancelación. Si Stripe rechaza el reembolso, la cancelación no se guarda y el viajero ve el error, así nunca queda una reserva cancelada sin su devolución.

Notificaciones: correo y campana

- **Correos**: `NotificationEmailService` arma el mensaje (verificación, reserva, pago, cancelación, recordatorio, recuperación) y lo envía por SMTP. Todos se guardan en la tabla **notificacion_email** con su estado: enviado, desactivado o fallido. Con `EMAILS_ENABLED=false` no sale ningún correo, pero queda el registro.
- **Correos con diseño**: `email_templates.render_email` arma cada correo en HTML con la marca, un recuadro de datos (código, fecha, total) y un botón. Se hace con tablas y estilos en línea porque Gmail y Outlook ignoran el CSS moderno. Cada correo se envía también en texto plano para los programas que no muestran HTML.
- **Campana**: `AppNotificationService.notify` guarda un aviso en **notificaciones_app** con título, mensaje y un enlace. El componente NotificationBell del frontend los consulta y muestra los no leídos.

Recordatorios automáticos

Al encender el servidor, `lifespan` lanza `reminder_loop`: un ciclo que cada hora busca reservas **confirmadas**, sin recordatorio enviado y cuyo viaje sea en los próximos 3 días. A cada una le manda correo y aviso en la campana, y marca `recordatorio_enviado` para no repetirlo.

backend/app/services/reminder_service.py (resumido)

```
async def reminder_loop(session_factory):
    while True:
        db = session_factory()
        try:
            ReminderService(db).send_due()
        finally:
            db.close()
        await asyncio.sleep(CHECK_EVERY_SECONDS)   # espera una hora
```

Fotos: Amazon S3 y CloudFront

Las fotos de paquetes y de perfil se suben a **Amazon S3** (un "disco" en la nube) con `core/storage.py`. Antes de subir se revisan los primeros bytes del archivo para confirmar que de verdad es una imagen (no basta con el nombre). Cada archivo recibe un nombre único y se sirve por **CloudFront**, que lo entrega rápido y lo guarda en caché.

Coordenadas automáticas de las ciudades

Cuando el administrador crea una ciudad sin latitud y longitud, `geocoding_service.find_coordinates` las pide al servicio gratuito Nominatim (OpenStreetMap). Así la ciudad aparece en el mapa de destinos sin buscar coordenadas a mano.

Voucher con QR, reportes y configuración

- **Voucher** (`voucher_service.py`): genera con reportlab un PDF de la reserva confirmada con un código QR. El QR abre la página pública **/verificar/código**, donde cualquiera puede comprobar que la reserva es real.
- **Reportes** (`report_service.py`): calcula ventas, reservas y pagos por mes en un rango de fechas y los exporta a Excel (openpyxl) o PDF (reportlab).
- **Configuración** (`setting_service.py`): días de cancelación gratis, porcentaje de penalidad, días para pagar a proveedores y días del recordatorio. Si un valor no está en la base, usa el valor por defecto de constants.py.
- **Otros servicios**: cupones, favoritos, reseñas, proveedores, productos, ciudades, eliminación segura de registros e inicio de sesión con Google.

CAPÍTULO 7

LA BASE DE DATOS

Toda la información vive en MySQL, en 21 tablas relacionadas entre sí con llaves foráneas.

Las 21 tablas agrupadas por tema

Tema	Tablas
Usuarios	usuarios, roles, clientes, proveedores
Catálogo	paquetes_turisticos, imagenes_paquetes, productos, paquetes_turisticos_has_productos, ciudades
Reservas y pagos	reservas, acompanantes, pagos, cupones
Proveedores	ordenes_pago_proveedor, ordenes_pago_detalle
Interacción	favoritos, resenas, notificaciones_app, notificacion_email
Administración	configuraciones, bitacora_admin

Cómo se relacionan las principales

- Un **usuario** tiene un **rol**. Si es cliente tiene un registro en **clientes**; si es proveedor, en **proveedores**.
- Un **paquete** pertenece a una **ciudad**, tiene varias **imágenes** e incluye varios **productos** de proveedores. La tabla **paquetes_turisticos_has_productos** une paquetes y productos (relación de muchos a muchos).
- Una **reserva** es de un **cliente** para un **paquete**; tiene **acompañantes**, puede tener un **pago** y un **cupón**.
- Al pagar una reserva se crean **órdenes de pago** para los proveedores, y cada orden tiene su **detalle** por producto.

Las migraciones

El script principal crea la base desde cero. Cuando el proyecto creció, en vez de rehacer la base se escribieron **migraciones**: scripts SQL pequeños que agregan solo lo nuevo, en orden:

Archivo	Qué agrega
migracion_001_google_usuarios.sql	Inicio de sesión con Google.
migracion_002_avatar_y_cancelaciones.sql	Foto de perfil y datos de cancelación.
migracion_003_pagos_proveedores.sql	Órdenes de pago a proveedores.
migracion_004_estado_envio_correos.sql	Estado de envío de cada correo.
migracion_005_mejoras.sql	Cupones, reseñas, favoritos, notificaciones, bitácora y configuración.
migracion_006_incluye_paquetes.sql	El campo “Qué incluye” de los paquetes.

CAPÍTULO 8

EL FRONTEND POR DENTRO

El frontend es una aplicación de React: una sola página que cambia de contenido sin recargar, y que pide los datos al backend a medida que los necesita.

Tres ideas de React para entender todo lo demás

Idea	Qué es	Ejemplo en Travelers
Componente	Una función que devuelve lo que se ve en pantalla. Se pueden anidar como bloques de Lego.	PackageCard dibuja una tarjeta de paquete.
Props	Los datos que un componente le pasa a otro, como los parámetros de una función.	<PackageCard pkg={paquete} />
Estado (useState)	Datos que el componente recuerda. Cuando cambian, React vuelve a dibujar la pantalla.	const [cantidadPersonas, setCantidadPersonas] = useState(1)
Efecto (useEffect)	Código que se ejecuta después de dibujar, por ejemplo para pedir datos al backend.	Cargar el paquete al abrir su página.

El punto de arranque: main.jsx

Todo empieza aquí. Se "envuelve" la aplicación en capas que le dan superpoderes a todo lo que está adentro:

frontend/travelers/src/main.jsx

```
ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <BrowserRouter>          {/* permite cambiar de pantalla con la URL */}
      <AuthProvider>         {/* comparte la sesión con toda la app */}
        <FavoritesProvider>  {/* comparte los favoritos */}
          <App />
        </FavoritesProvider>
      </AuthProvider>
    </BrowserRouter>
  </React.StrictMode>
);
```

Las pantallas: App.jsx

App.jsx dibuja siempre la barra de navegación y, en el medio, la pantalla que corresponde a la dirección. Cada `<Route>` une una dirección con un componente de página:

frontend/travelers/src/App.jsx (resumido)

```
<Routes>
  <Route path="/" element={<Home />} />
  <Route path="/packages" element={<Packages />} />
  <Route path="/packages/:id" element={<PackageDetail />} />
  <Route path="/reservations"
         element={<ProtectedRoute><Reservations /></ProtectedRoute>} />
  <Route path="/admin"
         element={<ProtectedRoute adminOnly><AdminDashboard /></ProtectedRoute>} />
  <Route path="/proveedor"
         element={<ProtectedRoute roles={[ROLE_PROVIDER]}><ProviderPanel /></ProtectedRoute>} />
  <Route path="*" element={<p>Página no encontrada.</p>} />
</Routes>
```

`:id` es un parámetro: en /packages/7 la página lee el 7 con `useParams()` y pide ese paquete al backend.

ProtectedRoute: el portero

frontend/travelers/src/components/ProtectedRoute/ProtectedRoute.jsx (resumido)

```
function ProtectedRoute({ children, adminOnly = false, roles }) {
  const { isAuthenticated, user, loadingUser } = useAuth();

  if (!isAuthenticated) return <Navigate to="/login" />;   // sin sesión
  if (loadingUser || !user) return <p>Cargando...</p>;     // esperando datos

  const allowedRoles = adminOnly ? [ROLE_ADMIN] : roles;
  if (allowedRoles && !allowedRoles.includes(user.rol_id))
    return <Navigate to="/" />;                             // rol equivocado

  return children;                                          // puede pasar
}
```

Hablar con el backend: axiosConfig.js

Todas las pantallas usan el mismo cliente `api`. Tiene la dirección del backend y un **interceptor**: una función que se ejecuta antes de cada petición y le pega el token automáticamente. Por eso ninguna pantalla tiene que acordarse de enviarlo.

frontend/travelers/src/api/axiosConfig.js

```
export const API_URL = (import.meta.env.VITE_API_URL
                        || 'http://127.0.0.1:8000').replace(/\/$/, '');

const api = axios.create({ baseURL: API_URL });

api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});
```

> **Nota.** En desarrollo el backend está en http://127.0.0.1:8000. En producción, al compilar se define `VITE_API_URL=https://travelers-sena.duckdns.org/api` y Vite escribe esa dirección dentro del código final.

La sesión: AuthContext

Un **contexto** es una forma de compartir datos con todos los componentes sin pasarlos de mano en mano. AuthContext guarda el token y el usuario:

1. Al iniciar sesión, `login(token)` guarda el token en el **localStorage** del navegador (sobrevive aunque se cierre la pestaña).
2. Cada vez que cambia el token, un `useEffect` llama a `GET /users/me` y guarda los datos del usuario (nombre, rol, foto).
3. Si esa petición falla (token vencido), borra el token: el usuario queda como visitante.

4. `logout()` borra el token y el usuario.
5. Cualquier componente obtiene la sesión con `const { user, logout } = useAuth()`.

Una pantalla por dentro: el formulario de reserva

En **PackageDetail.jsx**, cada campo del formulario tiene su estado (`fechaReserva`, `cantidadPersonas`, `companions`...). Al enviar:

frontend/travelers/src/pages/PackageDetail/PackageDetail.jsx

```
const handleSubmit = async (event) => {
  event.preventDefault();          // evita que el navegador recargue
  setFormError('');
  setSubmitting(true);             // desactiva el botón mientras tanto
  try {
    const response = await api.post('/reservations/', {
      id_paquete_turistico: touristPackage.id_paquete_turistico,
      fecha_reserva: fechaReserva,
      cantidad_personas: Number(cantidadPersonas),
      codigo_cupon: coupon ? coupon.codigo : undefined,
      acompanantes: companions.map(...),
    });
    navigate('/reservations', { state: { createdCode: response.data.codigo_reserva } });
  } catch (requestError) {
    setFormError(getErrorMessage(requestError, 'No fue posible crear la reserva'));
  } finally {
    setSubmitting(false);
  }
};
```

`getErrorMessage` (en utils/format.js) lee el campo `detail` que manda el backend. Así el mensaje que escribió el servicio, por ejemplo “No hay cupo suficiente. Cupos disponibles: 3”, llega tal cual a la pantalla.

Al abrir la página, en cambio, se piden varias cosas al mismo tiempo con `Promise.all` (fotos, cupos disponibles, ciudades, política de cancelación y qué incluye), para que cargue más rápido que pidiéndolas una por una.

Favoritos: respuesta inmediata

Al tocar el corazón, **FavoritesContext** cambia el ícono **de inmediato** y después avisa al backend. Si el backend falla, deshace el cambio. Esta técnica se llama “actualización optimista” y hace que la aplicación se sienta rápida.

Ayudantes reutilizables

Pieza	Para qué sirve
useAdminAction()	Ejecuta una acción de la consola y maneja el estado "ocupado", el mensaje de éxito y el de error. Evita repetir try/catch en cada botón.
useConfirm()	Muestra una ventana de confirmación con el estilo de la app y espera la respuesta: `if (!(await confirm({...}))) return;`
usePagination / Paged	Divide las listas largas en páginas de 10, 25 o 50.
usePackageCatalog()	Carga paquetes, portadas, ciudades y calificaciones y los une para las tarjetas.
Select y DatePicker	Listas desplegables y calendarios propios, accesibles con teclado; el calendario solo deja elegir fechas reservables.
downloadFile / openFile	Descargan o abren PDF y Excel protegidos enviando el token.
Toast y Modal	Mensajes flotantes y ventanas emergentes.

La consola de administración

AdminDashboard.jsx tiene una lista `SECTIONS` con las 15 secciones (resumen, paquetes, reservas, pagos, usuarios, reportes...). La sección activa se lee de la dirección (`/admin?seccion=reservas`), así se puede recargar o compartir el enlace sin perder la sección. Cada sección es un componente aparte dentro de `pages/Admin/sections/`.

El buscador del catálogo

Las descripciones no siempre usan la palabra que escribe el viajero: ningún paquete dice "playa", pero San Andrés dice "isla" y "Caribe". Por eso `utils/packageSearch.js` amplía cada búsqueda con **palabras relacionadas**:

frontend/travelers/src/utils/packageSearch.js (resumido)

```
const THEMES = {
  playa: ['playa', 'mar', 'caribe', 'isla', 'costa', 'tayrona', 'rosario', ...],
  cafe:  ['cafe', 'cafetero', 'finca', 'cocora', 'salento', 'jardin'],
  aventura: ['aventura', 'rafting', 'actividad', 'parapente', ...],
  // montaña, naturaleza, selva, cultura, historia, pueblo, fiesta...
};
```

- Quita tildes y mayúsculas, y compara por palabras completas aceptando plurales: "playas" funciona igual que "playa", y "mar" no encuentra "Marta".
- Busca en el nombre, la descripción, la ciudad, el departamento y lo que incluye el paquete.
- Debajo del buscador muestra "También buscamos: mar, caribe, isla..." para que se entienda cada resultado.
- **Filtros** que se combinan con el buscador: destino, mes, calificación (4.5 □, 4 □, 3 □, con reseñas), **con descuento** y orden (precio, próxima salida, mejor calificados, más reseñas). Todo queda en la dirección (por ejemplo `?q=playa&oferta=1`), así un enlace se puede compartir con los filtros puestos.

Paquetes relacionados y compartir

- **"También te puede gustar"**: al final del detalle, `relatedPackages` da puntos a los otros paquetes por compartir temas (playa, café, aventura…), estar en la misma región o tener precio parecido, y muestra los 3 mejores.
- **Botón Compartir**: WhatsApp, Facebook, correo y copiar enlace. En celulares agrega el menú para compartir del sistema (`navigator.share`).
- **Vista previa del enlace**: `index.html` tiene etiquetas Open Graph con una imagen de 1200×630, así al pegar el enlace en WhatsApp aparece la tarjeta con el logo. Cada pestaña tiene su propio título gracias al hook `usePageTitle`.

Carga más rápida

- **Carga por partes**: con `React.lazy` cada pantalla (la consola, el panel del proveedor, el mapa con Leaflet…) se descarga solo cuando se visita. El archivo principal bajó de 735 KB a 246 KB.
- **Siluetas de carga** (`components/Skeleton`): mientras llegan los datos se ven tarjetas grises animadas con la forma del contenido, en vez de un texto "Cargando…".

Los estilos

Cada componente tiene su propio archivo .css al lado. Los colores, fuentes y espaciados están definidos una sola vez como variables en **App.css** (`--color-primary`, `--color-surface`…), así cambiar un color de la marca se hace en un solo lugar. No se usó ninguna librería de componentes: todo está hecho a mano.

CAPÍTULO 9

RECORRIDO COMPLETO: DE UN CLIC A LA BASE DE DATOS

Uniendo todo lo anterior, esto es lo que pasa cuando un viajero pulsa “Reservar” en la página de un paquete.

1. **Navegador.** El viajero llena la fecha y las personas; React guarda cada valor en el estado del formulario.
2. **React.** Al pulsar “Reservar”, `handleSubmit` llama a `api.post('/reservations/', datos)`.
3. **Axios.** El interceptor agrega `Authorization: Bearer <token>` y envía la petición a https://travelers-sena.duckdns.org/api/reservations/.
4. **Nginx.** En el servidor recibe la petición por HTTPS y, como empieza por /api/, se la pasa al backend en el puerto 8000.
5. **Middlewares.** CORS confirma que la petición viene de la página permitida; la bitácora la deja pasar (no es un administrador).
6. **FastAPI.** Encuentra la ruta POST /reservations/, valida el JSON con `ReservationCreate` y ejecuta las dependencias: abre la sesión de MySQL y obtiene al usuario con `get_current_user`.
7. **Servicio.** Revisa perfil, paquete activo, fecha, acompañantes y cupo; calcula el total y aplica el cupón.
8. **Repositorio.** Inserta la fila en la tabla **reservas** con estado pendiente y hace commit.
9. **Efectos.** Se guardan los acompañantes, el correo de confirmación y el aviso de la campana.
10. **Respuesta.** FastAPI devuelve 201 con la reserva en JSON; Nginx la entrega al navegador.
11. **React.** Navega a “Mis reservas” y muestra el aviso con el código de la reserva. Si algo falló, muestra el mensaje del backend debajo del formulario.

> **Consejo.** Este mismo camino lo siguen las 122 rutas del API. Cambia la pantalla y el servicio, pero las capas y el orden son siempre los mismos. Entender este recorrido es entender casi todo el proyecto.

CAPÍTULO 10

RECETA: AGREGAR UNA FUNCIONALIDAD NUEVA

Gracias a las capas, agregar algo nuevo siempre sigue los mismos pasos. Por ejemplo, si se quisiera guardar "comentarios del viajero" sobre una reserva:

1. **Base de datos.** Escribir una migración nueva (migracion_007_...sql) que cree la tabla.
2. **Modelo.** Crear la clase en `app/models/` con sus columnas.
3. **Esquemas.** En `app/schemas/`, uno para los datos que llegan (Create) y otro para los que salen (Response).
4. **Repositorio.** Métodos para guardar y buscar.
5. **Servicio.** Las reglas: quién puede comentar, longitud máxima, que la reserva sea suya. Lanzar ValueError o PermissionError cuando no se cumplan.
6. **Ruta.** Un archivo en `app/routes/` con los endpoints, que traduzca los errores a códigos HTTP. Registrarlo en main.py con `include_router`.
7. **Prueba.** Escribir pruebas en `backend/test/` para los casos buenos y los que deben fallar.
8. **Frontend.** Llamar al endpoint con `api` desde la pantalla que corresponda, guardar la respuesta en el estado y mostrarla. Mostrar los errores con getErrorMessage.

CAPÍTULO 11

PRUEBAS AUTOMÁTICAS

El backend tiene 238 pruebas que se ejecutan con un solo comando y comprueban que las reglas del negocio se cumplen.

Cómo funcionan

- Usan una **base de datos de prueba** separada (DB_TEST_NAME), así nunca tocan los datos reales.
- **conftest.py** prepara todo: crea las tablas vacías antes de cada prueba, apaga correos, recordatorios y geocodificación (las pruebas no salen a internet) y reemplaza `get_db` por la base de prueba.
- Cada prueba usa un **cliente de prueba** de FastAPI que hace peticiones como si fuera el navegador, y luego revisa la respuesta.

backend/test/test_reservation_rules.py

```
def test_too_many_companions_does_not_leave_orphan_reservation(client, db_session):
    package, _ = create_package_ready_for_reservation(client, db_session)
    client_token, _ = create_client_with_token(client, db_session, ...)

    response = create_reservation(client, client_token,
                                  package_id=package["id_paquete_turistico"],
                                  cantidad_personas=1)

    assert response.status_code == 400                      # se rechaza
    assert db_session.query(ReservationModel).count() == 0   # y no queda nada
```

Para ejecutarlas, desde la carpeta backend: `pytest`. En el frontend, `npm run lint` revisa el código con oxlint.

CAPÍTULO 12

CÓMO SE PUBLICÓ EN INTERNET

La aplicación corre en un servidor de Amazon (EC2) con Ubuntu. Estas son las piezas y cómo se conectan.

DuckDNS travelers-sena.duckdns.org › **Nginx + HTTPS** Let's Encrypt › **/ → frontend** archivos de Vite › **/api/ → backend** uvicorn :8000 › **MySQL 8** en el mismo servidor

Pieza	Qué hace
EC2 (t3.micro, Ubuntu 24.04)	El computador en la nube donde corre todo, con una IP fija (elástica).
DuckDNS	Da el nombre travelers-sena.duckdns.org y lo apunta a la IP del servidor.
Nginx	Recibe todas las visitas. Entrega los archivos del frontend y pasa a uvicorn lo que empieza por /api/.
Certbot (Let's Encrypt)	Pone el candado HTTPS y renueva el certificado solo.
systemd (servicio travelers)	Mantiene el backend encendido y lo reinicia si el servidor se reinicia.
Archivo .env del servidor	Clave de los tokens, contraseña de MySQL y llaves de Stripe (STRIPE_SECRET_KEY y STRIPE_WEBHOOK_SECRET), con permisos solo para el dueño.
S3 + CloudFront	Guardan y entregan las fotos.
Stripe (modo de prueba)	Recibe los pagos y los reembolsos. Avisa a /api/payments/stripe/webhook cuando se completa un pago.

Cómo se actualiza

- **Frontend**: se compila con `npx vite build` (indicando VITE_API_URL) y se copian los archivos de la carpeta dist al servidor. Los archivos llevan un código en el nombre, por eso Nginx puede guardarlos en caché un año sin riesgo.
- **Backend**: se copia la carpeta app al servidor, se instalan las librerías nuevas si las hay y se reinicia con `sudo systemctl restart travelers`.
- **Base de datos**: se ejecuta en el servidor solo la migración nueva.

> **Importante.** El archivo .env y la llave .pem nunca se suben a GitHub: contienen claves que darían acceso al servidor y a la base de datos. Por eso están en el .gitignore.

Con este manual, las infografías técnicas y la documentación automática en /docs, se tiene el mapa completo del código de Travelers.